Experiment No: 2

Course Outcome: LO2 **Blooms Level: L3**

Aim: Implement N- Queen Uninformed Search Technique.

Abstract:

Place one queen in each row. If a position is safe, move to the next row. If not, try another column. If no column works, go back (backtrack) and try a different position.

The N-Queen Problem is one of the classical Artificial Intelligence search problems. The objective is to place N queens on an N × N chessboard such that no two queens attack each other. Since a queen can move horizontally, vertically, and diagonally, the algorithm must ensure that every newly placed queen satisfies these constraints. This experiment uses Depth First Search (DFS) with Backtracking, an uninformed search technique, to systematically explore possible configurations until a valid solution is found.

Problem Statement

Place N queens on an N × N chessboard such that:

- No two queens are placed in the same row.
- No two queens are placed in the same column.
- No two queens lie on the same diagonal.

The objective is to find one valid arrangement of all queens.

Sample Input:

N = 4

Theory:

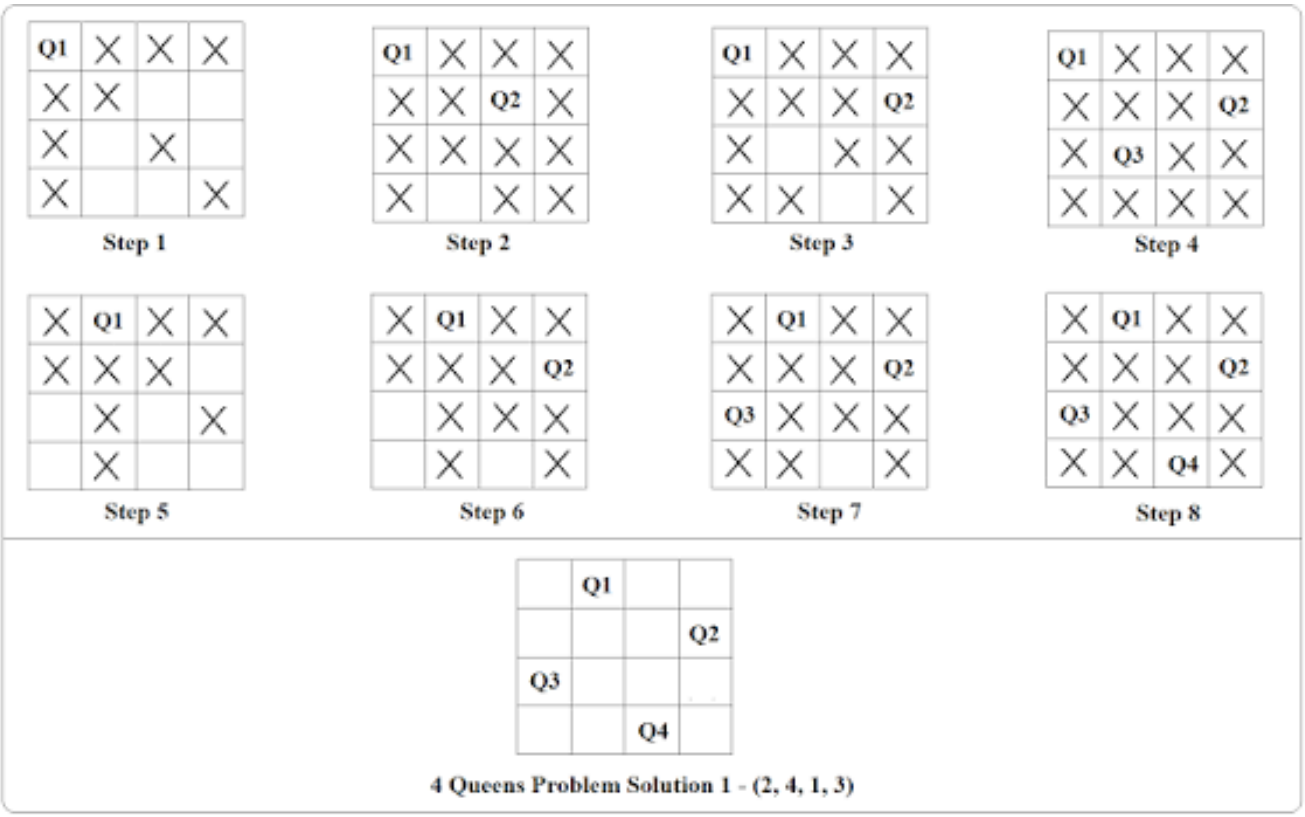
The N-Queen Problem is a well-known constraint satisfaction and state-space search problem in Artificial Intelligence. It demonstrates how a search algorithm explores different possible states until a solution satisfying all constraints is obtained.

This problem can be solved using Depth First Search (DFS) with Backtracking, which belongs to the category of Uninformed Search Algorithms because it does not use any heuristic information while searching.

The algorithm starts by placing the first queen in the first column. It then attempts to place another queen in the next column while ensuring that it is not attacked by previously placed queens.

If a safe position exists, the algorithm proceeds to the next column. If no safe position is available, the algorithm backtracks, removes the previously placed queen, and tries another possible position.

This process continues until all queens are successfully placed or all possible configurations have been explored.



Output:

Solution Found

0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

Where

1 = Queen
0 = Empty Cell

Conclusion:
This experiment helped in understanding state-space representation, constraint checking, DFS traversal, and backtracking. It illustrates how uninformed search algorithms can effectively solve classical Artificial Intelligence problems by exploring possible states without using heuristic information.

Exercise 1:
Modify the program to solve the 8-Queen Problem and display one valid solution.

Exercise 2:
Implement the Travelling Salesman Problem (TSP) for a graph of 4 cities using Depth First Search (DFS) and determine the minimum cost tour.

Class: TE Semester: V Subject: AISC

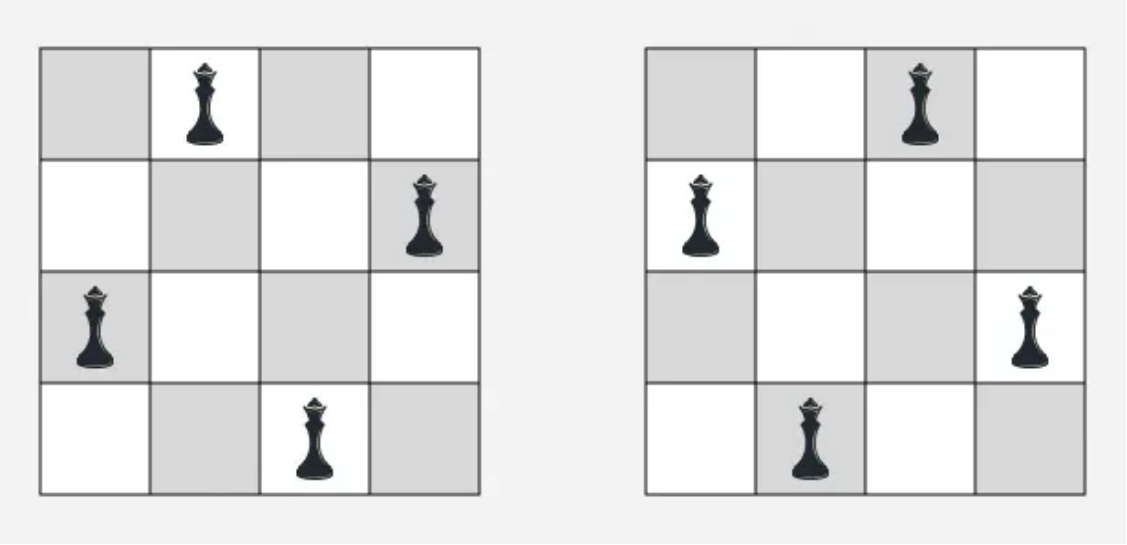
EXPLANATION (Self Reading)

N Queen Problem

Place **n queens** on an n×n chessboard so that no two attack each other (same row, column, or diagonal). Return all valid arrangements, where each solution shows the column position of the queen in each row.

Examples:

Input: *n = 4*
Output: *[[2, 4, 1, 3], [3, 1, 4, 2]]*
Explanation: *We mainly print column numbers (from first to last row) of every possible configuration.*



Input: *n = 3*
Output: *[]*
Explanation: *There are no possible solutions for n = 3*

Class: TE Semester: V Subject: AISC

Using Backtracking

Use backtracking to place queens row by row, checking if each position is safe. If safe, place the queen and move to the next row; otherwise, backtrack and try another position. Store the solution when all queens are placed.

